MODULE 15 · DOMAIN 3 - RESILIENT CLOUD SOLUTIONS

Scaling Everything

EC2 Auto Scaling policies, warm pools, lifecycle hooks and instance refresh; ECS and EKS scaling; Lambda concurrency; databases, RDS Proxy and queues.

DOMAIN 3 of 6 EXAM WEIGHT 15% MODULE 15 of 34

admin@sthithapragna.com

Every scaling question has three parts: the **signal** you scale on, the **speed** of the response, and the **ceiling** somewhere else that stops you.

What scales what

WORKLOAD	SCALES THE WORK	SCALES THE CAPACITY UNDERNEATH
EC2	Auto Scaling group policies	The group itself
Amazon ECS	Service auto scaling: task count	Capacity providers, or Fargate
Amazon EKS	Horizontal Pod Autoscaler: pods	Karpenter, Cluster Autoscaler, or Auto Mode
Lambda	Automatic, per request	Concurrency quotas and reservations
DynamoDB	On-demand, or provisioned with auto scaling	Managed by the service
Aurora	Replica auto scaling; Serverless v2 capacity	Managed by the service

ON THE EXAM

Containers have two layers. Scaling tasks or pods does nothing if there is no capacity to place them on.

EC2 Auto Scaling policies

POLICY	HOW IT DECIDES	BEST FOR
Target tracking	Holds a metric near a target	Most workloads
Step scaling	Bigger steps as alarms breach further	Fine control
Simple scaling	One step, then a cooldown	Legacy
Scheduled	Sets capacity at set times	Known events
Predictive	Forecasts and scales ahead	Daily or weekly patterns

ON THE EXAM

Target tracking plus predictive scaling covers most real workloads: react to surprises, prepare for patterns.

Target tracking, done well

- Choose a metric that rises in proportion to load: CPU, ALB requests per target, or a custom metric.
- With several target tracking policies, the group scales **out if any** wants to, and **in only if all** agree.
- **Default instance warm-up** stops new instances' low metrics from triggering scale-in too early.
- CPU is the wrong signal for I/O-bound or queue-driven work. Scale on what actually hurts.

> ON THE EXAM
>
> Scaling reacts, but on the wrong thing: change the metric, not the threshold.

Predictive scaling

- Learns daily and weekly patterns from up to 14 days of history, then launches capacity ahead of forecast demand.
- Start in **forecast only** mode to compare the forecast with reality, then switch to **forecast and scale**.
- A **pre-launch buffer** starts instances early enough to finish booting before the peak.
- Combine with target tracking, which handles whatever the forecast missed.

> ON THE EXAM
>
> A peak at the same time every weekday, and target tracking lags: predictive scaling, not hand-maintained schedules.

Warm pools: for applications that boot slowly

- A **warm pool** keeps pre-initialised instances ready: stopped, hibernated, or running.
- Scale-out moves an instance from the pool into service, skipping the slow initialisation.
- Stopped instances cost only their storage. Hibernated ones keep memory, so caches survive.
- A reuse policy can return instances to the pool on scale-in instead of terminating them.

ON THE EXAM

Minutes-long start-up makes scale-out too slow for spikes: a warm pool, rather than running peak capacity all day.

Lifecycle hooks

- Pause an instance while **launching** or **terminating**, so you can act before it moves on.
- The hook emits an EventBridge event; a Lambda function or SSM runbook does the work.
- Finish with **complete-lifecycle-action**, or extend with heartbeats. Default timeout one hour.
- Typical uses: register with a service before traffic; copy logs or drain work before termination.

> ON THE EXAM
>
> Copy logs before scale-in terminates an instance: a terminating lifecycle hook with an SSM command.

Instance refresh

- Replaces instances gradually after a launch template or AMI change.
- **Minimum healthy percentage** keeps capacity; **checkpoints** pause at chosen percentages.
- **Skip matching** leaves instances already on the new template alone.
- **Automatic rollback** returns to the previous template on failure, or when a CloudWatch alarm fires.

> ON THE EXAM
>
> Roll a new AMI across the group, keep 90% healthy, pause to check, roll back on alarms: instance refresh.

Health, Spot and instance choice

- Turn on **ELB health checks** so the group replaces instances the load balancer marks unhealthy.
- A **health check grace period** stops new instances being replaced before they finish starting.
- **Mixed instances** policies combine On-Demand and Spot across many instance types.
- **Capacity rebalancing** replaces Spot instances proactively when they are at risk.

> ON THE EXAM
>
> Instances keep failing and relaunching right after start: the grace period is shorter than the start-up time.

Scaling Amazon ECS

- **Service auto scaling** changes the task count: target tracking on CPU, memory, or ALB requests per target.
- **Capacity providers** scale the EC2 instances underneath, with managed scaling.
- A capacity provider strategy mixes Fargate and Fargate Spot, with a base and weights.
- Tasks stuck in PROVISIONING mean the task count scaled but the capacity did not.

> ON THE EXAM
>
> Queue workers: scale on backlog per task, not CPU. That needs a custom metric.

Scaling queue workers on backlog per task

```
# Each task processes about 60 messages a minute.
# Messages must be processed within 5 minutes of arrival.
# So an acceptable backlog is 60 x 5 = 300 messages per task.

backlog  = SQS ApproximateNumberOfMessagesVisible
tasks    = ECS RunningTaskCount
metric   = backlog / tasks          # CloudWatch metric math

target tracking: keep metric at 300
```

3 The target comes from the service level objective, not from guesswork.

7 Metric math divides one metric by another, with no custom code.

ON THE EXAM

Backlog alone does not scale well: 1,000 messages means different things for 2 tasks and for 50.

Scaling Amazon EKS

- The **Horizontal Pod Autoscaler** changes pod counts; **KEDA** scales pods on events such as queue depth.
- **Cluster Autoscaler** resizes node groups, but is slow and packs pods poorly across many groups.
- **Karpenter** launches right-sized nodes directly, uses Spot, and consolidates underused nodes.
- **EKS Auto Mode** (December 2024) runs Karpenter-style node management for you.

> ON THE EXAM
>
> Faster node scaling, better packing, Spot, least operational overhead: EKS Auto Mode.

Lambda concurrency

SETTING	WHAT IT DOES	COST
Account concurrency	Shared per Region; 1,000 by default	None
Scaling rate	1,000 new environments per function per 10 s	None
Reserved concurrency	Guarantees a share, and caps it	None
Provisioned concurrency	Pre-initialised: no cold starts	While configured
SnapStart	Restores from a cached snapshot	Low; not with provisioned

ON THE EXAM

429s below the account limit: the scaling rate was exceeded, or another function took the pool.

Reserved or provisioned concurrency?

PROVISIONED

How ready

- Environments initialised in advance
- No cold starts, up to the number set
- Set on a version or alias
- Scale it on a schedule with Application Auto Scaling

RESERVED

How much

- Guarantees concurrency for this function
- Also caps it: protects downstream systems
- Free
- Zero stops the function entirely

THE TIEBREAKER

Starved by other functions, or overwhelming a database: reserved. Cold starts hurting latency: provisioned or SnapStart.

Lambda and its event sources

- With SQS, Lambda adds pollers as the queue grows. **Maximum concurrency** on the mapping caps it.
- Batch size and batching window trade latency for efficiency.
- **Partial batch responses** retry only the failed messages, not the whole batch.
- Set the queue's visibility timeout well above the function timeout, or messages are processed twice.

> ON THE EXAM
>
> Limit how fast a queue drains into a fragile database: maximum concurrency on the event source mapping.

Scaling databases

- **Read replicas** absorb read traffic; Aurora can add and remove replicas automatically.
- **Aurora Serverless v2** scales capacity in fine steps, and since November 2024 can pause at zero.
- **DynamoDB on-demand** needs no capacity planning; provisioned mode can auto scale.
- Caching with **ElastiCache** or **DAX** removes reads before they reach the database.

> ON THE EXAM
>
> Writes do not scale with read replicas. A write bottleneck needs a bigger writer, sharding, or a different store.

RDS Proxy

- **Pools and reuses** database connections, so bursts of clients share a few connections.
- Ideal for Lambda, where each concurrent environment would otherwise open its own connection.
- Supports IAM authentication and Secrets Manager credentials.
- Speeds up failover, because clients stay connected to the proxy.

> ON THE EXAM
>
> 'Too many connections' as Lambda scales: RDS Proxy. A bigger instance only moves the ceiling.

> An ECS service processes messages from an SQS queue. Each task spends most of its time waiting on a slow partner API, so CPU stays under 20%.
>
> The service scales on CPU, and during bursts the backlog grows for hours while task count stays flat. Messages must be processed within 10 minutes.

What should the DevOps engineer change?

A Lower the CPU target to 10%

B Add scheduled scaling for the busiest hours

C Double the CPU and memory of each task

D Target tracking on backlog per task: visible messages divided by running tasks, using metric math

◆ PAUSE AND DECIDE

What should the DevOps engineer change?

A **Lower CPU target**

CPU does not rise with the backlog, so it stays a poor signal at any target.

B **Scheduled scaling**

Bursts are not described as predictable, and it still ignores the backlog.

C **Bigger tasks**

Tasks are waiting on an API, not short of CPU. More CPU changes nothing.

D **Backlog per task**

Scales on what actually hurts, with a target derived from the 10-minute objective.

THE SIGNAL

Wrong signal: the thing that hurts is the backlog. Scale on backlog per task.

> Several Lambda functions share an account. Every night a batch function scales to most of the account's concurrency.
>
> During that time, the customer-facing API function is throttled with 429 errors. The API needs about 200 concurrent executions at night.

What should the DevOps engineer do?

A Request a higher account concurrency quota, and change nothing else

B Set reserved concurrency of 200 on the API function

C Configure provisioned concurrency on the batch function

D Enable SnapStart on the API function

◆ PAUSE AND DECIDE

What should the DevOps engineer do?

A. Bigger quota only

The batch function can grow to consume the new quota too.

B. Reserve 200 for the API

Guarantees the API its share, whatever the batch function does. Free.

C. Provisioned on batch

Makes the batch function faster to start, and adds cost. It still takes the pool.

D. SnapStart on the API

Removes cold starts. Throttling is about concurrency, not start-up.

THE SIGNAL

One function starving another: reserved concurrency. Capping the batch function would also work.

> An application on an EC2 Auto Scaling group takes about eight minutes to start, while it loads a large cache. Traffic spikes are sudden and unpredictable.
>
> Scale-out is too slow to protect users during spikes. The company wants faster scale-out without paying for peak capacity all day.

What should the DevOps engineer do?

A Run the group at peak capacity at all times

B Replace target tracking with predictive scaling

C Add a warm pool of pre-initialised instances in the stopped or hibernated state

D Move to larger instance types

◆ PAUSE AND DECIDE

What should the DevOps engineer do?

A **Peak capacity always**

Fast, but pays for peak all day: ruled out.

B **Predictive scaling**

Needs patterns. Spikes here are unpredictable, and instances still take eight minutes.

C **Warm pool**

Instances have already initialised, so scale-out skips the eight minutes. Stopped ones cost only storage.

D **Larger instances**

The cache still takes minutes to load.

THE SIGNAL

Too slow because start-up is slow: pre-initialise. Warm pools do it cheaply.

An API runs on Lambda and writes to an Aurora PostgreSQL cluster. During traffic bursts, Lambda scales to hundreds of concurrent executions.

Requests start failing with 'too many connections' errors, although the database's CPU and memory are fine.

What should the DevOps engineer do?

A Put Amazon RDS Proxy between the functions and the database

B Move the database to a larger instance class

C Add an Aurora read replica

D Enable SnapStart on the function

◆ PAUSE AND DECIDE

What should the DevOps engineer do?

A RDS Proxy

Pools connections, so hundreds of environments share a few real connections.

B Larger instance

Raises the connection limit a little. The next burst hits it again.

C Read replica

The API writes. Replicas only take reads.

D SnapStart

Speeds start-up. Each environment still opens its own connection.

THE SIGNAL

CPU fine, connections exhausted, Lambda in front: the ceiling is connections. RDS Proxy.

> A booking site on an EC2 Auto Scaling group sees traffic triple every weekday at 09:00, and the pattern has been stable for months. Target tracking reacts, but users see slow pages for the first ten minutes.
>
> The team wants capacity in place before the rise, with the least ongoing maintenance, and still wants protection against unexpected surges.

What should the DevOps engineer do?

A. Create scheduled actions for 08:45 each weekday, and review them monthly

B. Add a predictive scaling policy in forecast and scale mode, and keep the target tracking policy

C. Add a warm pool

D. Replace target tracking with aggressive step scaling

◆ PAUSE AND DECIDE

What should the DevOps engineer do?

A Scheduled actions

Works, but must be maintained by hand as the pattern shifts. More upkeep.

B Predictive plus target tracking

Learns the pattern, launches ahead of 09:00, and target tracking still catches surprises.

C Warm pool

Speeds each launch, but still waits for the rise to start before scaling.

D Aggressive step scaling

Still reactive: always behind a rise that has already begun.

THE SIGNAL

Recurring pattern, capacity before the rise, least maintenance: predictive scaling alongside target tracking.

A platform team publishes a patched AMI and updates an Auto Scaling group's launch template. The group runs 40 instances behind a load balancer.

Instances must be replaced gradually while keeping 90% healthy, pausing after 20% for checks, and rolling back automatically if an error-rate alarm fires.

Which solution meets these requirements with the LEAST effort?

A Terminate instances by hand in batches of four

B Create a second Auto Scaling group and move the load balancer to it

C A CodeDeploy blue/green deployment of the new AMI

D An instance refresh with 90% minimum healthy, a checkpoint at 20%, and automatic rollback on the alarm

◆ PAUSE AND DECIDE

Which solution meets these requirements with the LEAST effort?

A Manual termination

No checkpoint, no automatic rollback, and hours of human effort.

B Second group

A whole second fleet and a manual switch, with no gradual replacement.

C CodeDeploy blue/green

Deploys application revisions, not AMIs. The wrong tool for an image change.

D Instance refresh

Every requirement is a setting: minimum healthy, checkpoint, alarm rollback.

THE SIGNAL

New AMI in a launch template, gradual, checkpointed, alarm rollback: instance refresh.

> Instances in an Auto Scaling group write important application logs to local disk, uploading them to S3 every hour. When the group scales in, up to an hour of logs is lost.
>
> The team wants the remaining logs copied to S3 before each instance terminates, with the least effort. The application is not deployed with CodeDeploy.

What should the DevOps engineer do?

A. A terminating lifecycle hook, with an EventBridge rule that runs an SSM command to copy the logs, then completes the lifecycle action

B. Add a shutdown script to the instances' user data

C. Increase the load balancer's deregistration delay

D. Enable scale-in protection on every instance

◆ PAUSE AND DECIDE

What should the DevOps engineer do?

A Terminating lifecycle hook

Pauses termination, runs the copy, then releases the instance. Exactly its purpose.

B User data

User data runs at launch, not at termination.

C Deregistration delay

Lets requests finish. It does not run anything on the instance.

D Scale-in protection

Stops scale-in altogether, which defeats scaling.

THE SIGNAL

Act on an instance before it terminates, without CodeDeploy: a terminating lifecycle hook.

> An EKS cluster uses the Cluster Autoscaler with eleven managed node groups. Nodes take several minutes to appear, pods are packed poorly, and Spot is barely used.
>
> The team wants faster node scaling, right-sized nodes, more use of Spot, and the least operational overhead.

What should the DevOps engineer do?

A. Add more managed node groups with more instance types

B. Tune the Horizontal Pod Autoscaler's thresholds

C. Enable EKS Auto Mode for the cluster

D. Move every workload to Fargate profiles

◆ PAUSE AND DECIDE

What should the DevOps engineer do?

A More node groups

More groups make the Cluster Autoscaler's packing problem worse, not better.

B Tune the HPA

Scales pods. The problem is nodes.

C EKS Auto Mode

Managed, Karpenter-style nodes: fast, right-sized, Spot-aware, consolidated, with no node groups to run.

D Fargate for everything

Removes nodes, but DaemonSets and some workloads cannot run there, and Spot is lost.

THE SIGNAL

Node scaling slow, packing poor, Spot unused, least overhead: Auto Mode, or Karpenter if you run it yourself.

Module 15 in five lines

- Signal, speed, ceiling: find which one failed. Scale on what actually hurts.
- Target tracking for most, predictive for patterns, warm pools for slow starts.
- Lifecycle hooks act before launch or termination; instance refresh rolls new AMIs with rollback.
- Containers scale twice: tasks or pods, then capacity. Auto Mode for EKS nodes.
- Lambda: reserved for how much, provisioned for how ready. RDS Proxy for connection ceilings.

ON THE EXAM

Next, module 16: disaster recovery. RTO and RPO, four strategies, Backup, Elastic Disaster Recovery and FIS.